Knowledge Based Chat bot

T Raghuveer

Mentor : Prof Harish Karnick

Introduction & Motivation

- We have an ever **increasing world of services** that use data from diverse sources.
- Many systems are being built that can intelligently respond to users who use **chat apps**/whatsapp in offering **variety of services**.
- The services can range over vertical segments like
  - Travel (road, rail, air)
  - House hold services (like plumbers, electricians etc.)
  - Restaurant/food services
  - Queries or a request for an actual bill payments, house deliveries.

Introduction & Motivation

- Many chat based service providers do this currently with **human agents** with relatively weak systems support.
- With **increase in number of users**, it becomes essential that these chat services become **automated**.
- We in this work try to create one such chat bot which can interact with the user

Knowledge based ??

Some existing chat bots which are in use

- Google now and Siri , cortana
- Some other LSTM/NN based bot models

We wanted a bot for our use case

- Knowledge based
- Dynamically updating knowledge

Problem statement

- Design system **scalable and robust** in incorporating new, upcoming verticals without disturbing the existing systems.
- Read, understand and extract **topic** content available in the text message.
- Query a **database or a web API** for content required to address the user's request or query.
- Ensure a smooth flow of conversation keeping a balance between **interaction and information**.

Data Preprocessing

- Chat data provided by yana
  - Data received in form of a raw Excel dump containing messages from multiple users intermixed with one another.
  - The delimiter tab and indentation tab were not properly differentiated leading to lot of confusion
- Data Preprocessing
  - This raw data was pre-processed to extract multiple transactions based on timestamp, user.id and context
  - Basic Keywords used to categorise them
  - converted into a more oraganised representation (XML)

Chat Example


```
<Message A="1" time="7/25/2015 8:21">Hi</Message>
<Message A="0" time="7/25/2015 8:22">Could check me a ticket availability to London by the end of august</
Message>
<Message A="0" time="7/25/2015 8:22">In economy class what ever possibility is there you can book</Message>
<Message A="0" time="7/25/2015 8:50">If possible check for some good flight</Message>
<Message A="0" time="7/25/2015 8:51">Flying from Chennai or banglore or Hyderabad to London Heathrow airport</
Message>
<Message A="0" time="7/25/2015 8:51">On my name</Message>
<Message A="0" time="7/25/2015 8:51">Date is on Sept 3</Message>
<Message A="1" time="7/25/2015 10:20">Hi. Here are some options as per your itinerary-</Message>
<Message A="1" time="7/25/2015 10:20">whatsappdata/image/whatsapp_img_38181437762600.png</Message>
<Message A="0" time="7/25/2015 10:20">Ok go on</Message>
<Message A="0" time="7/25/2015 10:20">You check from directly Delhi to london</Message>
<Message A="1" time="7/25/2015 10:21">Give me a moment to check and get back</Message>
<Message A="1" time="7/25/2015 10:24">You are looking for a direct flight\, right?</Message>
<Message A="0" time="7/25/2015 10:24">Yep</Message>
<Message A="1" time="7/25/2015 10:24">Sure.</Message>
<Message A="1" time="7/25/2015 10:25">whatsappdata/image/whatsapp_img_66781437762600.png</Message>
<Message A="0" time="7/25/2015 10:27">Go for British airways</Message>
<Message A="1" time="7/25/2015 10:27">Sure. Please give me a moment.</Message>
```

Methodology

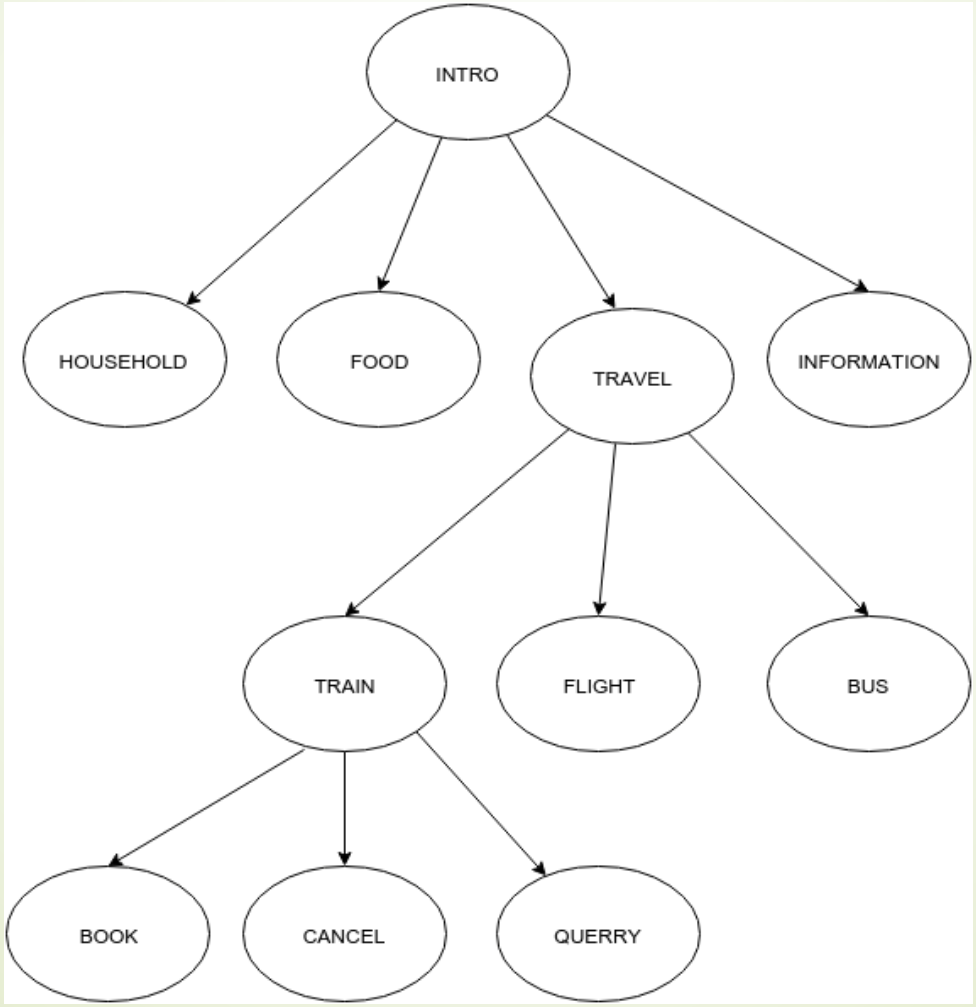
Our System mainly consists of two components

- **Central Architecture**
- **Template, Fillers Transition makers and Nodal Querries**

- Central Architecture is common for all the verticals.
- Data pickers, Querries, transition States have to be changed for each vertical

Central Architecture

State transition system

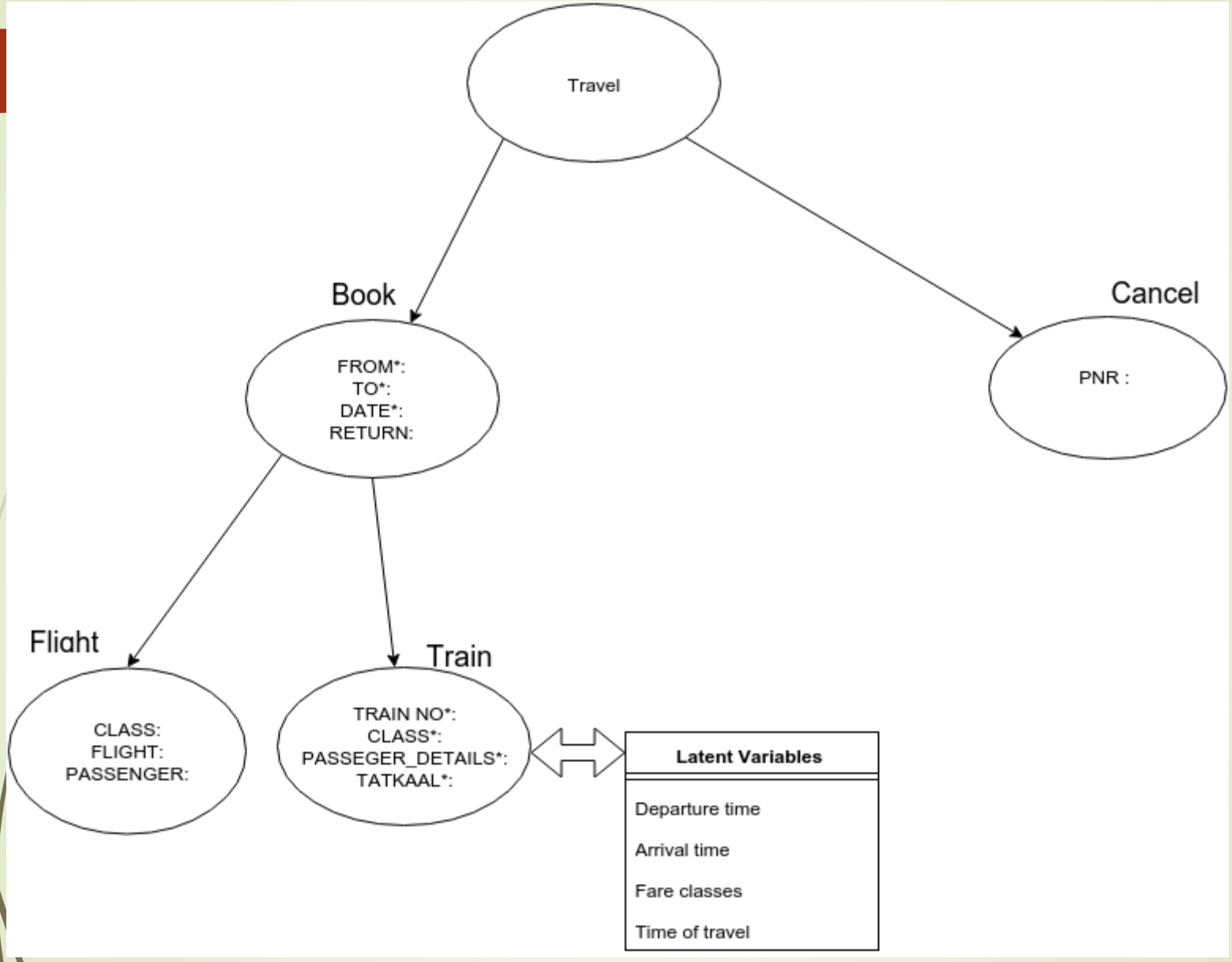


Node Structure

Every node consists of a set of variables-(Functional and Latent) and some functions.

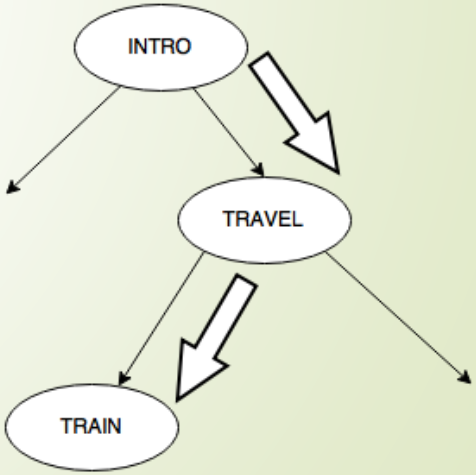
- *Functional variables*
  - These are information variables you expect to be present at the time of transition out of the node.
  - A single transaction ends when all the FV's are from root leaf path are filled up
- *Latent Variables*
  - those variables on which other functional variables may depend.

- Ex: 'Give me the cheapest flight from bangalore to delhi' cheapest here paves the way to fill up some functional variables.
  - Fare here is latent
  - Flight here is functional


Travel
Book
Cancel
FROM*:
TO*:
DATE*:
RETURN:
PNR :
Flight
Train
CLASS:
FLIGHT:
PASSENGER:
TRAIN NO*:
CLASS*:
PASSEGER_DETAILS*:
TATKAAL*:
Latent Variables
Departure time
Arrival time
Fare classes
Time of travel

Transition markers, Template Fillers and Nodal Querries

- Transition markers :
  - Responsible for **change of state of conversation**
  - Works essentially based on keywords
- Ex:
  - ‘Book me a train ticket from delhi to lucknow’



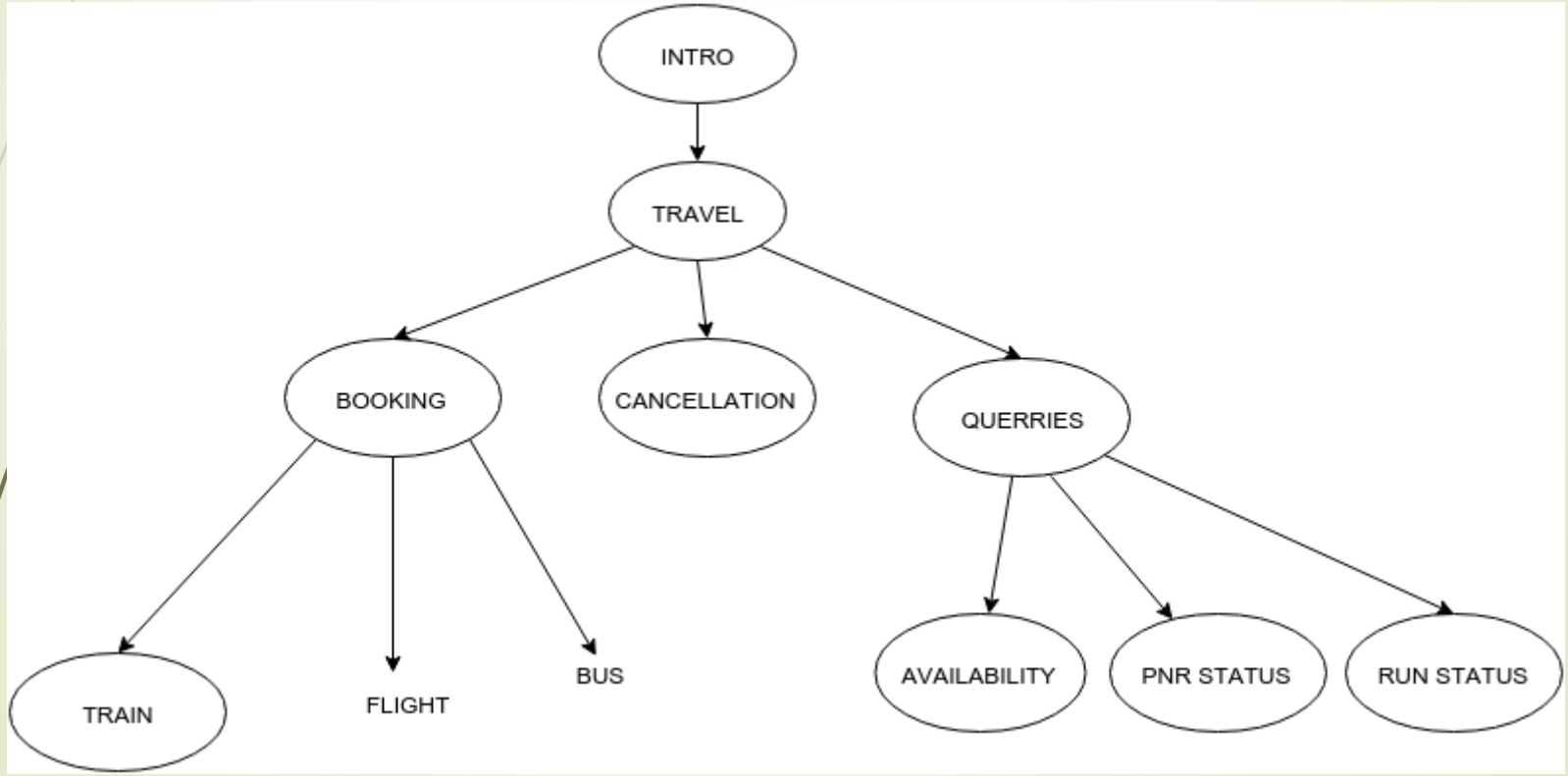
Template Fillers & Nodal Querries

- Template Fillers
  - Used to pick up functional variables from the chat by analysing text.
  - Example : date_picker picks date out of a chat
    - 'book a ticket on 12th dec' => 12/10/2015

- Nodal Querries
  - Operate over latent variables, and appropriately generate replies to questions related to latent variables.
  - answer some mid transaction queries, one or more functional variables are initialized
  - Ex :
    - A train berth availability querry between two stations, to decide upon the train number
    - A fare querry to decide upon the class of travel

Implementation

- We have the working system implemented in vertical of travel
- A 9 node working system



Some important functions

- Station Source destination picker
  - List of all stations was scraped
  - A bloat set including spelling errors for each word found
  - Soundex was used to pick the best similar sounding words
- Given a chat turn, pick **n grams** and **fuzzy match** with the bloated set and the max matching words are picked
- These words are then labelled into **source or destination** by means of some **hand coded pattern set** which can evolve

Other functions

- Train name picker
- Date picker
- Class picker
- Train no picker
- Passenger details
  - Name, Age , Sex pickers
- Running status Querry
- Availability Querry
- PNR status Querry
- Train between stations Querry *

Further Work

- To integrate the source destination - train cross check with train availability for more intelligent train booking conversations.
- To build a mechanism to automatically incorporate unseen structures with minimal human interference.
- Modelling of a conversation as multiple transactions rather than a single transaction.

Demo

Thank you

Suggestions/ Doubts ?